Natural Computing SoSe24

Late Exam* — October 16th, 2024

Please mind the following:

- Fill in your **personal information** in the fields below.
- You may use an unmarked dictionary during the exam. **No** additional tools (like calculators, e.g.) might be used.
- Fill in all your answers **directly into this exam sheet**. You may use the backside of the individual sheets or ask for additional paper. In any case, make sure to mark **clearly** which question you are answering. Do not use pens with green or red color or pencils for writing your answers. You may give your answers in both **German and English**.
- Points annotated for the individual tasks only serve as a preliminary guideline.
- At the end of the exam hand in your **whole exam sheet**. Please make sure you do not undo the binder clip!
- To forfeit your exam ("entwerten"), please cross out clearly this **cover page** as well as **all pages** of the exam sheet. This way the exam will not be evaluated and will not be counted as an exam attempt.
- Please place your LMU-Card or student ID as well as photo identification ("Lichtbildausweis") clearly visible on the table next to you. Note that we need to check your data with the data you enter on this cover sheet.

Time Limit: 90 minutes

First Name:	
Last Name:	
Matriculation Number:	

Topic 1	max. 13 pts.	pts.
Topic 2	max. 12 pts.	pts.
Topic 3	max. 19 pts.	pts.
Topic 4	max. 10 pts.	pts.
Topic 5	max. 14 pts.	pts.
Topic 6	max. 14 pts.	pts.
Topic 7	max. 8 pts.	pts.
Total max. 90 pts.		pts.

*Corrections to the original exam are marked in red in this file.

1 Game of Life **13pts**

The following definition was given in the lecture.

Definition 1 (Conway's game of life (standard))**.** Let $G = (V, E)$ be a grid graph with vertices V and (undirected) edges $E \subseteq V \times V$ with a fixed degree of 8 for all nodes. We define $\mathit{surroundings} : V \to \wp(V)$ via

$$\mathit{surroundings}(v) = \{w \mid (v, w) \in E\}$$

so that $|\mathit{surroundings}(v)| = 8$ and $v \notin \mathit{surroundings}(v)$ for all $v \in V$. A state $x \in \mathcal{X}$ is a mapping of vertices to the labels $\{\mathit{dead}, \mathit{alive}\}$, i.e., the state space $\mathcal{X}$ is given via $\mathcal{X} = (V \to \{\mathit{dead}, \mathit{alive}\})$. Let x_t be a state that exists at time step $t \in \mathbb{N}$. We define

$$|v|_{x_t} = |\{w \mid w \in \mathit{surroundings}(v) \wedge x_t(w) = \mathit{alive}\}|.$$

In the game of life, the evolution of a state x_t to its subsequent state x_{t+1} is given deterministically via

$$x_{t+1}(v) = \begin{cases} \mathit{dead} & \text{if } |v|_{x_t} \leq 1, \\ x_t(v) & \text{if } |v|_{x_t} = 2, \\ \mathit{alive} & \text{if } |v|_{x_t} = 3, \\ \mathit{dead} & \text{if } |v|_{x_t} \geq 4, \end{cases}$$

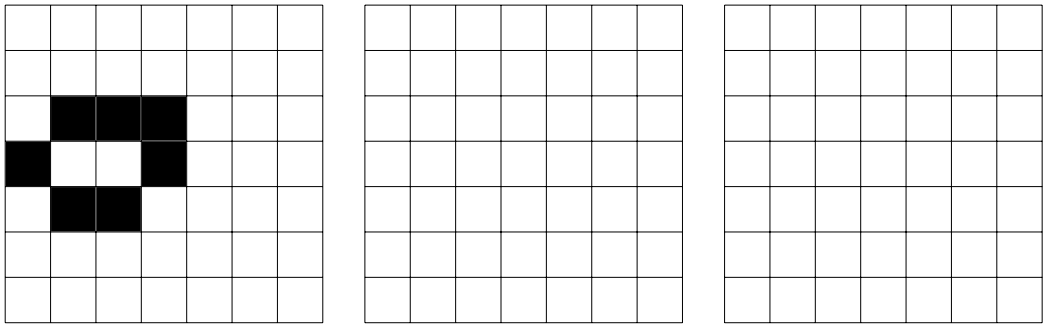
for all $v \in V$. A tuple (G, x_0) is called an instance of the game of life for initial state $x_0 \in \mathcal{X}$.

While the so called *Moore neighborhood* as defined in Def. 1 is commonly used to play Conway's Game of Life, there exists a multitude of different neighborhood functions. One such lesser known function is the *von Neumann neighborhood*, which only considers the 4 directly surrounding neighbor cells, i.e., to the top, right, bottom, and left of the cell v.

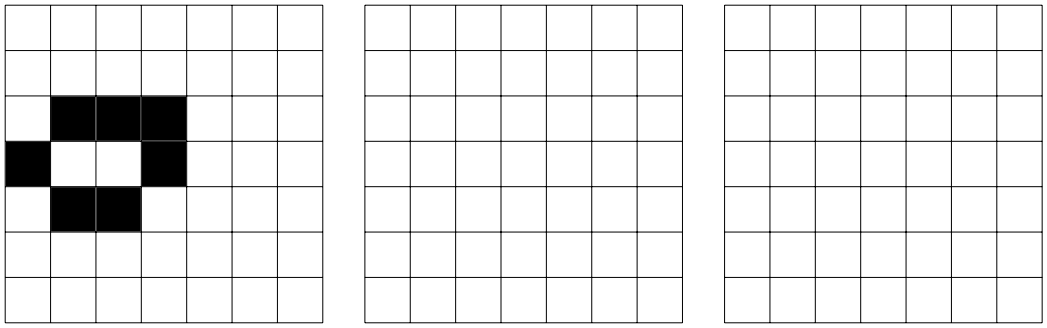
(a) **State which part of Def. 1 we would need to change to enable an evolution with the *von Neumann neighborhood* and give the updated part of the definition (not the whole text).** (2pts)
Hint: We only need to update a minor part of the definition.

(b) Below, the initial state to a pattern called *honey farm* is given. **Continue evolving the pattern in the game of life for <u>two</u> more steps, first with the *Moore* neighborhood and then again with the *von Neumann* neighborhood.** (8pts)

With *Moore* neighborhood:



With *von Neumann* neighborhood:



<u>Note</u>: There is free space in the Appendix at the back of the exam, should you need more than the one extra grid space printed here. Mark clearly on the task's page, if the solution to a task is to be found there, and mark clearly in the Appendix, which task a solution belongs to.

(c) **Thinking more generally, give an argument which of the two neighborhood functions (*Moore* or *von Neumann*) is more likely to exhibit larger patterns in the $B3/S23$ ruleset. Give your reasoning.** (3pts)

2 Cellular Automata **12pts**

The following definitions (Defs. 2 and 3) were given in the lecture.

Definition 2 (cellular automaton)**.** Let $G = (V, E)$ be a graph with vertices V and edges $E \subseteq V \times V$. Let *neighborhood* $: V \to V^{d+1}$ for some neighborhood degree $d \in \mathbb{N}$ be a function that returns an ordered vector of neighbors of a given node v, always including v itself. A state $x \in \mathcal{X}$ is a mapping of vertices to the values $\{0, 1\}$, i.e., the state space $\mathcal{X}$ is given via $\mathcal{X} = (V \to \{0, 1\})$. Let x_t be a state that exists at time step $t \in \mathbb{N}$. The evolution of a state x_t to its subsequent state x_{t+1} is given deterministically via a function $f : \{0, 1\}^{d+1} \to \{0, 1\}$ so that

$$x_{t+1}(v) = f\big(x_t(u_1), \ldots, x_t(u_{d+1})\big)$$
$$\text{where } \vec{u} = \langle u_1, ..., u_{d+1}\rangle = \textit{neighborhood}(v).$$

A tuple (G, f, x_0) is called a cellular automaton with initial state $x_0 \in \mathcal{X}$.

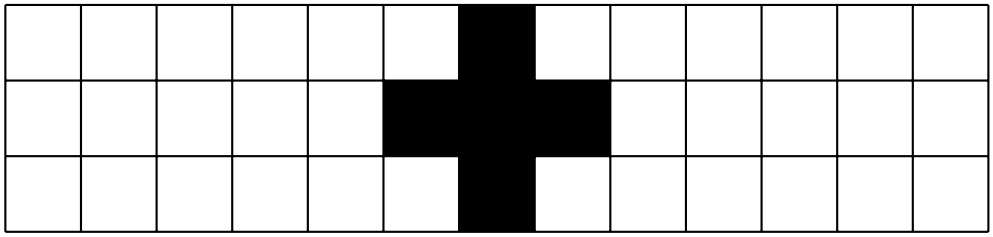
Definition 3 (1D cellular automaton)**.** A cellular automaton (G, f, x_0) is called a 1D cellular automaton iff all vertices have exactly one incoming edge ("left neighbor") and one outgoing edge ("right neighbor") and *neighborhood*$(v) = \langle u, v, w\rangle$ where $(u, v) \in E$ and $(v, w) \in E$ and thus $d = 2$. Subsequently, we usually write

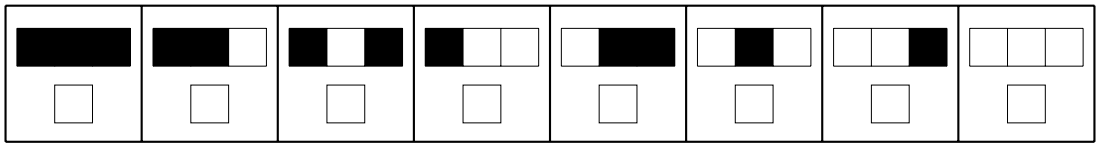
$$x_{t+1}(v) = f\big(x_t(u), x_t(v), x_t(w)\big)$$

for the evolution where u is the left neighbor of v and w is the right neighbor of v.

A 1D cellular automaton is often given via a *rule* $n \in [0; 255] \subset \mathbb{N}$ where each digit in n's binary representation encodes one output of f so that the digit for the base value 128 encodes the output of $f(1, 1, 1)$, 64 for $f(1, 1, 0)$, and so on, sorted by decreasing numerical value.

(a) **Given the following evolution, fill in the rule template below as completely as possible. Briefly state whether the given evolution contains enough information to identify the rulestring in question. Then state the integer of any possible rule(s) that you have marked in the template.** (5pts)





We know want to define a *weighted 1D automaton* (G, f', x_0, p) for $p \in [0; 1] \subset \mathbb{R}$ based on Defs. 2 and 3 with the following properties:

- The computation of $x_{t+1}(v)$ from x_t now depends on the weighted sum of the values of the neighborhood of v. That weighted sum is called $W_t(v)$.
- For that weighted sum $W_t(v)$, v's left neighbor is weighted with p, v is weighted with 1, and v's right neighbor is weighted with p^2.

(b) **Write down the adjusted formula for f' for the *weighted 1D automaton* $(G, f', x_0, 0.5)$ so that for a cell v it holds that $x_{t+1}(v) = 1$ if and only if the neighborhood of v has a weighted sum greater than 1.0.** (4pts)

(c) Let us assume we use a *weighted 1D automaton* as we just defined, but with base weight $p = 0.0$. **What is the expected behavior of the resulting automaton compared to using a *weighted 1D automaton* with $p = 0.5$?** (3pts)

3 Quantum Computing **19pts**

The following definitions were given in the lecture.

Definition 4 (quantum state)**.** Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{Q}(\mathcal{X})$ be the space of superposition states with bases in $\mathcal{X}$, i.e., each $q \in \mathcal{Q}(\mathcal{X})$ has the form $q = \sum_{x\in\mathcal{X}} c_x |x\rangle$ where $c_x \in \mathbb{C}$ for any $x \in \mathcal{X}$ is called an amplitude so that $\sum_{x\in\mathcal{X}} |c_x|^2 = 1$ and $|x\rangle$ (read "ket x") denotes a vector corresponding to the state $x \in \mathcal{X}$.

We use the following terminology:

- If for more than one $x \in \mathcal{X}$ it holds that $c_x \neq 0$, then q is called a *superposition* state.
- If $\mathcal{X} = \mathcal{X}_A \times \mathcal{X}_B$, i.e., $\mathcal{X}$ can be constructed as a product of two other state spaces $\mathcal{X}_A, \mathcal{X}_B$, then it follows that q has the form

  $$q = \sum_{\substack{x_A\in\mathcal{X}_A,\\ x_B\in\mathcal{X}_B}} c_{x_A,x_B} |x_A\rangle \otimes |x_B\rangle$$

  where $\otimes$ is the tensor product. If q cannot be written as

  $$q = \big(\sum_{x_A\in\mathcal{X}_A} c_{x_A} |x_A\rangle\big) \otimes \big(\sum_{x_B\in\mathcal{X}_B} c_{x_B} |x_B\rangle\big),$$

  then we call q an *entangled* state.
- If $\mathcal{X} = \{0, 1\}$ (i.e., any $x \in \mathcal{X}$ is a bit), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit* and can be written as

  $$q = c_0 |0\rangle + c_1 |1\rangle$$

  for $c_0, c_1 \in \mathbb{C}$ with $|c_0|^2 + |c_1|^2 = 1$.
- If $\mathcal{X} = \{0, 1\}^n$ for any $n \in \mathbb{N}$ (i.e., any $x \in \mathcal{X}$ is a bit register), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit register* of size n and can be written as

  $$q = \sum_{i=0}^{2^n-1} c_i |i\rangle = c_{0\ldots0} |0\ldots0\rangle + \cdots + c_{1\ldots1} |1\ldots1\rangle$$

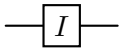
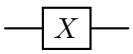
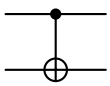
  where we usually denote i in binary as illustrated above.
- When we *measure* a quantum state $q \in \mathcal{Q}(\mathcal{X})$, we generate classical state $M(q) \in \mathcal{X}$ so that for all $x \in \mathcal{X}$ it holds that

  $$\Pr(M(q) = x) = |c_x|^2.$$

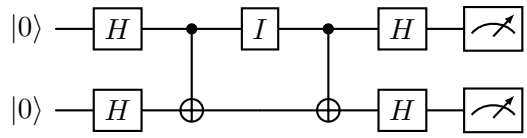
(a) **Complete the quantum state $|a\rangle = 0.5 \cdot |101\rangle + o$ by giving the omitted term o such that $|a\rangle$ is a valid quantum state. Show your reasoning.** (4pts)

(b) **State if $|b\rangle = |001001\rangle$ is in an entangled quantum state or not. Show your reasoning.** (3pts)

For the following tasks remind yourself of these common quantum operators as we have covered in the lecture:

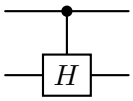
Operator	Gate	Matrix
Identity (I)		$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$
Hadamard (H)		$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
Pauli-X (X)		$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Controlled-X (CNOT)		$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$

(c) For the circuit below, we know that the resulting state after being measured will always be $|00\rangle$. **Briefly explain how we can arrive at this insight without computing the whole ciruit.** (3pts)



(d) **Considering the quantum operators and their matrix representations in the table above, give the $\mathbb{R}^{4\times4}$ matrix operation of a *Controlled-Hadamard* gate, which should work for H like a CX gate works for X, i.e., the H gate is executed on the second qubit insofar the first qubit is in the state $|1\rangle$.** (3pts)

Controlled-Hadamard (CH)



(e) **Using <u>one</u> of the above quantum operators and the <u>Controlled-Hadamard gate</u> as explained in task (d) above, construct a quantum circuit that prepares the following state $|abc\rangle$ by drawing the operator gates on the qubit wire template below. Briefly explain your answer.** (6pts)
Note: We draw the leftmost qubit in the state notation as the topmost wire.

Prepare the state:

$$|abc\rangle = \frac{1}{\sqrt{2}}|100\rangle + \frac{1}{\sqrt{2}}|101\rangle$$

$|a\rangle := |0\rangle$ ——————————

$|b\rangle := |0\rangle$ ——————————

$|c\rangle := |0\rangle$ ——————————

4 (Population-Based) Optimization 10pts

We give a shortened version of the definition for optimization processes from the lecture and then introduce a new optimization algorithm called *artificial bee colony algorithm*.

Definition 5 (optimization (shortened))**.** Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{T}$ be an arbitrary set called target space and let $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \to \mathcal{T}$ is called target function. Optimization (minimization/maximization) is the procedure of searching for an $x \in \mathcal{X}$ so that $\tau(x)$ is optimal (minimal/maximal). Unless stated otherwise, we assume minimization.

An optimization run of length $g+1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t.

Let $e : \langle \mathcal{X} \rangle \times (\mathcal{X} \to \mathcal{T}) \to \mathcal{X}$ be a possibly randomized or non-deterministic function so that the optimization run $\langle x_t \rangle_{0 \leq t \leq g}$ is produced by calling e repeatedly, i.e., $x_{t+1} = e(\langle x_u \rangle_{0 \leq u \leq t}, \tau)$ for all t, $1 \leq t \leq g$, where x_0 is given externally (e.g., $x_0 =_{def} 42$) or chosen randomly (e.g., $x_0 \sim \mathcal{X}$). An optimization process is a tuple $(\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$.

Definition 6 (population-based optimization)**.** Let $\mathcal{X}$ be a state space. Let $\mathcal{T}$ be a target space with total order $\leq$. Let $\tau : \mathcal{X} \to \mathcal{T}$ be a target function. A tuple $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \tau, E, \langle X_t \rangle_{0 \leq t \leq g})$ is a population-based optimization process iff $X_t \in \wp^*(\mathcal{X})$ for all t and $E : \langle \wp^*(\mathcal{X}) \rangle \times (\mathcal{X} \to \mathcal{T}) \to \wp^*(\mathcal{X})$ is a possibly randomized, non-deterministic, or further parametrized function so that the population-based optimization run is produced by calling E repeatedly, i.e., $X_{t+1} = E(\langle X_u \rangle_{0 \leq u \leq t}, \tau)$ where X_0 is given externally or chosen randomly.

For this exercise we imagine a high-dimensional optimization algorithm emulating the foraging behavior of honey bees, modeled via a population $X_t = \{x_{i,t}\}_{i=1}^{n}$ representing the positions of n bees (solutions) in the search space $\mathcal{X}$ at step t.

Algorithmically, such an *artificial bee colony algorithm*[1] could consist of two phases:

1. **Employed Bee Phase**: Each bee i explores around its current solution by generating a new candidate solution using the formula:

   $$v_{i,t} = x_{i,t} + 0.1 \cdot (x_{i,t} - x_{k,t})$$

   If $\tau(v_{i,t}) > \tau(x_{i,t})$, then evolve $x_{i,t+1} = v_{i,t}$. Here, $x_{k,t}$ is a randomly selected solution *different* from our current bee (solution) $x_{i,t}$, i.e., $x_{k,t} \sim X_t \setminus \{x_{i,t}\}$.

[1] adapted from Karaboga, D. (2005)

2. **Scout Bee Phase**: If a bee i was not successful with its search after a certain number of steps, the solution is abandoned and a new random solution $z_{i,t}$ is generated within the search space:

$$z_{i,t} = x_{i,t} + \texttt{srandom()} \cdot (x_{k_1,t} - x_{k_2,t}),$$

where $\texttt{srandom()} \sim [-1;1] \subset \mathbb{R}$ is a random sampling function and $x_{k_1,t}, x_{k_2,t}$ are, again, *different* solutions to $x_{i,t}$, i.e., $x_{k_1,t} \sim X_t \setminus \{x_{i,t}\}$ and $x_{k_2,t} \sim X_t \setminus \{x_{i,t}, x_{k_1,t}\}$. We can check if a bee i is still considered to have had recent success via a given predicate function $\mathit{had\_success}_t : \mathbb{N} \to \mathbb{B}$. This means: If $\neg \mathit{had\_success}_t(i)$, then evolve $x_{i,t+1} = z_{i,t}$.

(a) **Complete the formal definition of the artificial bee colony optimization (maximization) below by giving a suitable evolution function E in accordance to the steps outlined by the artificial bee colony algorithm above. You can use all variables as defined above.** (8pts)

Algorithm 1 (artificial bee colony optimization)**.** Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \tau, E, \langle X_t \rangle_{0 \leq t \leq g})$ be a population-based optimization process. We assume $\mathcal{X} = [-10;10]^d \subset \mathbb{R}^d$ contains solutions $x \in \mathcal{X}$ with d dimensions and $\mathcal{T} = [0;100] \subset \mathbb{R}$ for maximization. Let $\tau(x)$ represent any suitable target function. For $x \notin [-10;10]^d$, we assume that we can still call τ so that $\tau(x) = -\infty$. The process $\mathcal{E}$ continues via artificial bee colony optimization iff E is given via:
$E(\langle X_u \rangle_{0 \leq u \leq t}, \tau) = X_{t+1} =$

where $X_t = \{x_{i,t}\}_{i=1}^n$ are the positions of n bees (solutions) in search space $\mathcal{X}$ at step t, $\mathit{had\_success}_t : \mathbb{N} \to \mathbb{B}$ is a suitable predicate function for a bee i and $x_{k,t}, x_{k_1,t}, x_{k_2,t}$ are different solutions to $x_{i,t}$, i.e., $x_{k,t}, x_{k_1,t} \sim X_t \setminus \{x_{i,t}\}$ and $x_{k_2,t} \sim X_t \setminus \{x_{i,t}, x_{k_1,t}\}$.

(b) **State whether the artificial bee colony algorithm could be considered *greedy*. Briefly explain your reasoning.** (2pts)

5 Artificial Chemistries / Soups **14pts**

Recall the following definition from the lecture:

Definition 7 (artificial chemistry, soup)**.** Let $\mathcal{X}$ be a state space. Let $\mathcal{R}$ be a set of reaction rules $R \in \mathcal{R}$ where each R is a function $R : \mathcal{X}_R^{\text{dom}} \to \mathcal{X}_R^{\text{cod}}$ for $\mathcal{X}_R^{\text{dom}}, \mathcal{X}_R^{\text{cod}} \subseteq \wp^*(\mathcal{X})$. Let $\mathcal{A}(X, \mathcal{R})$ be the set of all *reaction materials* from $X \in \wp^*(\mathcal{X})$ for all rules in $\mathcal{R}$, i.e.,

$$\mathcal{A}(X, \mathcal{R}) = \left\{(X', R) \;\middle|\; X' \subseteq X \;\wedge\; X' \in \mathcal{X}_R^{\text{dom}} \;\wedge\; R \in \mathcal{R}\right\}.$$

Let $A : \wp^*(\mathcal{X}) \to \wp^*(\mathcal{X}) \times \mathcal{R}$ be a possibly randomized or non-deterministic function that takes a multi-set $X \in \wp^*(\mathcal{X})$ and returns a multi-set of reactants and a reaction rule to perform so that $A(X) \in \mathcal{A}(X, \mathcal{R})$. The tuple $(\mathcal{X}, \mathcal{R}, A)$ defines an artificial chemistry. A multi-set $X \in \wp^*(\mathcal{X})$ of an artificial chemistry is also called soup; each element $x \in X$ is also called a particle. The evolution of a soup for g generations is a sequence $\langle X_0, .., X_g \rangle$ so that X_0 is given as the initial soup and, for all t with $0 \leq t \leq g - 1$

$$X_{t+1} = (X_t \setminus X') \uplus R(X')$$

where $(X', R) = A(X_t)$.
Reaction rules are usually written in the form $R = \sum_i x_i \rightharpoonup \sum_j y_j$ for $x_i, y_j \in \mathcal{X}$, $i, j \in \mathbb{N}$, $\{x_i\}_i \in \mathcal{X}_R^{\text{dom}}, \{y_j\}_j \in \mathcal{X}_R^{\text{cod}}$, and $R(\{x_i\}_i) = \{y_j\}_j$.

Note: We use the following (very intuitive) shortcut in notation: The expression $n \cdot x$ for some $n \in \mathbb{N}$ and $x \in \mathcal{X}$ is defined to be $\underbrace{x + \ldots + x}_{n \text{ times}}$.

We now define the artificial chemistry $\mathcal{C} = (\mathcal{X}, \mathcal{R}, A)$ by defining:

Let $\mathcal{X} = \{\textsc{SearchingBee}(c), \textsc{ReturningBee}(c), \textsc{CelebratingBee}(c), \textsc{Flower}(c), \textsc{Hive}(n)\}$ for all $c \in C$ where $C = \{\textsc{Red}, \textsc{Blue}, \textsc{Yellow}\}$ and all $n \in \mathbb{N}$.

Let $\mathcal{R} = \{R_{1,\_,\_}, R_{2,\_}, R_{3,\_,\_}, R_{4,\_,\_}\}$ with

$$\begin{aligned}
R_{1,c,d} &= \textsc{SearchingBee}(c) + \textsc{Flower}(d) &&\rightharpoonup \textsc{ReturningBee}(c) + \textsc{Flower}(d)\\
R_{2,c} &= \textsc{SearchingBee}(c) + \textsc{Flower}(c) &&\rightharpoonup \textsc{CelebratingBee}(c) + \textsc{Flower}(c)\\
R_{3,c,c'} &= \textsc{CelebratingBee}(c) + \textsc{ReturningBee}(c') &&\rightharpoonup \textsc{ReturningBee}(c) + \textsc{ReturningBee}(c')\\
R_{4,c,n} &= \textsc{ReturningBee}(c) + \textsc{Hive}(n) &&\rightharpoonup \textsc{SearchingBee}(c) + \textsc{Hive}(n+1)
\end{aligned}$$

for all $c, c' \in C, d \in C \setminus \{c\}$.

Let $A(X) \sim \mathcal{A}(X, \mathcal{R})$.

Intuitively, we consider a soup made out of bees in various behavioral states. Each bee has a favorite color. If it finds a flower of that color, it celebrates before returning to the hive; from other flowers, a bee returns immediately. Celebrating bees only stop when they see another bee returning to the hive and are thus reminded of their job. The hive keeps count of how much pollen has been brought back by returning bees.

(a) **Give a possible evolution of the soup**

$$X_0 = \{\textsc{Hive}(2), \textsc{SearchingBee}(\textsc{Red}), \textsc{ReturningBee}(\textsc{Blue}), \textsc{Flower}(\textsc{Blue})\}$$

for as many steps as necessary until $\textsc{Hive}(5)$ **appears, i.e., until for the current soup** X_t **at some time** t **it holds that** $\textsc{Hive}(5) = \textsc{Hive}(2+1+1+1) \in X_t$**.** (7pts) (Hint: Instead of random choices, you can decide which rules to apply to make the process quicker. Use the table below.)

time step	chosen rule	current soup
0	none	$\{\textsc{Hive}(2), \textsc{SearchingBee}(\textsc{Red}),$ $\textsc{ReturningBee}(\textsc{Blue}), \textsc{Flower}(\textsc{Blue})\}$
1		
2		
3		
4		
5		
6		
7		

(b) A beekeeper friend (whose bees also implement the artificial chemistry $\mathcal{C}$) told us that their bees have stopped working. They observed that for their population of bees X'_t at time t it holds that $G \subset X'_t$ for

$$G = \{\textsc{CelebratingBee}(\textsc{Red}), \textsc{CelebratingBee}(\textsc{Blue}), \\ 2 \cdot \textsc{CelebratingBee}(\textsc{Yellow}), \textsc{Hive}(42)\}.$$

Give the <u>minimal</u> (multi-)set H so that $G \uplus H = X'_t$ and so that X'_t could develop from a soup X'_0 that contained no $\textsc{CelebratingBee}(\_)$ **according to $\mathcal{C}$.** (3pts)

(c) To fix their problem with celebrating bees, our beekeeper friend wants to introduce one additional particle to the soup X'_t, i.e., set $X'_{t+1} = X'_t \uplus \{x\}$ for some $x \in \mathcal{X}$ and then continue (for computing X'_{t+2}, X'_{t+3}...) according to the rules of $\mathcal{C}$. **What particle x might be added that could help with this problem? Given that $\mathcal{C}$ is well stirred and all applicable rule instances are executed as given by A, is there a possibility that adding said particle fails to keep the bees from celebrating? Show your reasoning.** (4pts)

6 Evolutionary Computing **14pts**

If necessary, you can recollect the definition for evolutionary algorithms as used in the lecture on this page. (Hint: Try the tasks first before reading this lengthy definition again.)

Algorithm 2 (typical evolutionary algorithm)**.** Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be a population-based optimization process.

- Let $\sigma_N^{survivors}, \sigma_N^{parents} : \wp^*(\mathcal{X}) \to \wp^*(\mathcal{X})$ be (possibly randomized or non-deterministic) selection functions that return $N \in \mathbb{N}$ individuals, i.e., $|\sigma_N(X)| = N$ and $\sigma_N(X) \subseteq X$ for all X. They may possibly depend on $\mathcal{E}$ (most commonly ϕ). Let $\varrho_N^{mutants} : \wp^*(\mathcal{X}) \to \wp^*(\mathcal{X})$ be a special selection function that, given a population X, returns a random subset $X' \subseteq X$ so that $|X'| = N$. Note that $\varrho_{|X|}(X) = X$ for all X.

- Let $mutate : \mathcal{X} \to \mathcal{X}$ be a (possibly randomized or non-deterministic) single-individual mutation function. We write $mutate[\_] : \wp^*(\mathcal{X}) \to \wp^*(\mathcal{X})$ for the per-individual application of $mutate$, i.e., $mutate[X] = \{mutate(x) \ : \ x \in X\}$.

- Let $recombine : \mathcal{X} \times \mathcal{X} \to \mathcal{X}$ be a (possibly randomized or non-deterministic) two-parent recombination function. We write $recombine[\_] : \wp^*(\mathcal{X}) \to \wp^*(\mathcal{X})$ for the random-pairing application of $recombine$, i.e., $recombine[X] = \{recombine(x_1, x_2)\} \uplus recombine[X \setminus \{x_1, x_2\}]$ with $x_1, x_2 \sim X$ and $recombine[\{x\}] = \emptyset$ for all x as well as $recombine[\emptyset] = \emptyset$.

- Let $migrate : \emptyset \to \mathcal{X}$ be a (possibly randomized or non-deterministic) individual creation function. We write $migrate_N : \emptyset \to \wp^*(\mathcal{X})$ with $N \in \mathbb{N}$ for the iterated application of $migrate$, i.e., $migrate_N[\,] = \{migrate()\} \uplus migrate_{N-1}[\,]$ with $migrate_0[\,] = \emptyset$.

The process $\mathcal{E}$ continues via a typical evolutionary algorithm if E has the form

$$\begin{aligned} E(\langle X_u \rangle_{0 \leq u \leq t}, \phi) = X_{t+1} = \sigma_{|X_t|}^{survivors}\big(X_t \\ \uplus\ & mutate[\varrho_M^{mutants}(X_t)] \\ \uplus\ & recombine[\sigma_{2C}^{parents}(X_t)] \\ \uplus\ & migrate_H[\,]\big) \end{aligned}$$

where $M \in \mathbb{N}$ is the number of mutants produced per generation, $C \in \mathbb{N}$ is the number of children produced per generation, and $H \in \mathbb{N}$ is the number of migrants (sometimes called hypermutants) generated per generation. Especially M is often expressed as a rate for random choice within the population, i.e., $M = \lceil m \cdot |X_t| \rceil$ where $m \in \mathbb{R}$ is called mutation rate.

We now want to build an artificial bee robot, *beebot* for short. The beebot can discern exactly 1000 different types of flowers and, for each type, it has a certain preference. Thus, a beebot's behavior is given by a vector $x \in \mathcal{X}$ with

$$\mathcal{X} = \{(x_0, x_1, ..., x_{999}) \in \mathbb{R}_+^{1000} \mid \sum_{i=0}^{999} x_i = 1\}.$$

To optimize the beebots' preference vector, we use an evolutionary algorithm with a population size of 100.

(a) We recognize that simulating the beebot's behavior in a practical scenario usually requires using a swarm of them. To save computing time for evaluation, we decide to evaluate the whole population at once by simulating 100 beebots, one for each individual in the population, in a jointly interactive virtual environment. We then count how much pollen each beebot managed to bring back to the hive and use that metric as the fitness value for that beebot's preference vector. **Briefly explain a potential problem with choosing that approach for evaluation.** (3pts)

(b) For the recombination function, we consult our beekeeper friend who is not very experienced with evolutionary algorithms. They implement one-point crossover, but we end up generating lots of incorrect individuals, i.e., child vectors which do not fall in $\mathcal{X}$ because their values in each dimension do not sum up to 1. **State the likely problem that is occurring when using one-point crossover without any adaptation to match the state space $\mathcal{X}$ and briefly describe a possible approach to solve that problem.** You do not need to give working implementation of that approach. (4pts)

(c) A beautiful mutation function, which can be called via $mutate(x)$ on a solution candidate $x \in \mathcal{X}$, has the following properties:

- The generated mutant differs only slightly from the parent.
- The values of each dimension in the parent have an equal chance of being changed by the mutation.
- Through arbitrarily many mutation operations, any value $x^\dagger \in \mathcal{X}$ can be reached starting from any value $x \in \mathcal{X}$.
- The generated mutant x' is still a valid individual, i.e., $x' \in \mathcal{X}$.

Give a beautiful mutation function. (7pts)

Note that you can use a function `random()`, which returns a random floating point number between 0.0 (inclusive) and 1.0 (exclusive), i.e., $[0.0, 1.0)$, and you can use the notation $x \sim X$ to sample from a finite set X.

7 Mix-Up: Soups for Optimization **8pts**

Consider the evolutionary algorithm defined in task 6. We now define a soup made up of swarms of beebots. For this, we again use the population of an evolutionary algorithm as one swarm of 100 beebots. In our new artificial chemistry $\mathcal{C}^\dagger = (\mathcal{X}^\dagger, \mathcal{R}^\dagger, A^\dagger)$, we consider such a population to be one single particle. We encode this via the following definitions:

Let $\mathcal{X}^\dagger = \left\{(x_0, x_1, ..., x_{999}) \in \mathbb{R}_+^{1000} \mid \sum_{i=0}^{999} x_i = 1\right\}^{100}$.
Let $\mathcal{R}^\dagger = \{R^\dagger_{\_,\_,\_,\_}\}$.
Let $A^\dagger(X) \sim \mathcal{A}(X, \mathcal{R}^\dagger)$.

(a) In the artificial chemistry $\mathcal{C}^\dagger$, we expect the following behavior when two arbitrary particles $x, y \in X_t$ meet: x and y exchange exactly one of their individuals, i.e., one individual from the population represented by x is moved over to the population represented by y, and vice versa. This leaves the amount of individuals per population as well as the amount of particles in the soup X_t the same. **Give the rule(s)** $R^\dagger_{\_,\_,\_,\_}$ **so that this behavior is implemented.** (5pts)

(b) Let us assume that the search space $\mathcal{X}$ of our evolutionary algorithm, i.e., the space of all preference vectors, is characterized by having a large number of deceptive local optima. We now generate 10 initial populations of the evolutionary algorithm (EA) described in task 6. We perform a study comparing two independent scenarios: (A) We run these 10 initial populations of the EA independently. (B) We run one soup of $\mathcal{C}^\dagger$ as described above with these 10 initial populations of the EA as particles. **In which of these scenarios should we expect to see a higher diversity of the found solution candidates? State your reasoning.** (3pts)

Appendix.
On this page, you find additional grid fields if you need them for the tasks in this exam. Mark clearly on the task's page, if the solution to a task is to be found here, and mark clearly on this page here, which task a solution belongs to.